JOB APPLICATION TRACKER

1. Project Overview

The **Job Application Tracker** is a console-based Python application developed using Object-Oriented Programming (OOP) and modular programming concepts. The application allows users to manage profiles, job listings, and job applications while validating eligibility based on GPA and required skills. It uses JSON files for data storage, custom exceptions for validation, and recursive job recommendations based on partial skill matching.

2. Problem Statement

Develop a modular Python application that manages user profiles, job listings, and job applications. The application should support CRUD operations, validate job eligibility based on GPA and required skills, store data using JSON files, handle errors using custom exceptions, and recommend suitable jobs using a recursive function based on partial skill matching.

3. Features

User Profile Management

- Create user profiles
- View user profiles
- Update user profiles
- Delete user profiles

Job Management

- Add job listings
- View available jobs
- Update job details
- Delete job listings

Application Processing

- Apply for jobs

- Validate minimum GPA
- Validate required skills
- Store successful applications with date and time
- View submitted applications

Job Recommendations

- Recommend jobs based on partial skill matching
- Use recursion for job recommendations

Data & Error Handling

- Store data using JSON files
- Handle custom exceptions
- Modular code organization

4. Technologies Used

- Python 3
- Visual Studio Code
- JSON
- Object-Oriented Programming
- Modular Programming
- Command Prompt / Terminal

5. Python Concepts Used

- Object-Oriented Programming (OOP)
- Classes and Objects
- Constructors (__init__)
- Modular Programming
- JSON File Handling
- File Handling
- Custom Exceptions
- Exception Handling

- CRUD Operations
- Lists
- Dictionaries
- Loops
- Conditional Statements
- User Input Handling
- Datetime Module
- Menu-Driven Programming
- Recursion

6. Program Workflow

Start

↓

Create / Manage User Profile

↓

Create / Manage Job Listings

↓

Apply for a Job

↓

Validate GPA and Required Skills

↓

Store Application with Date and Time

↓

Recommend Jobs Using Partial Skill Matching

↓

Use Recursive Function to Check Available Jobs

↓

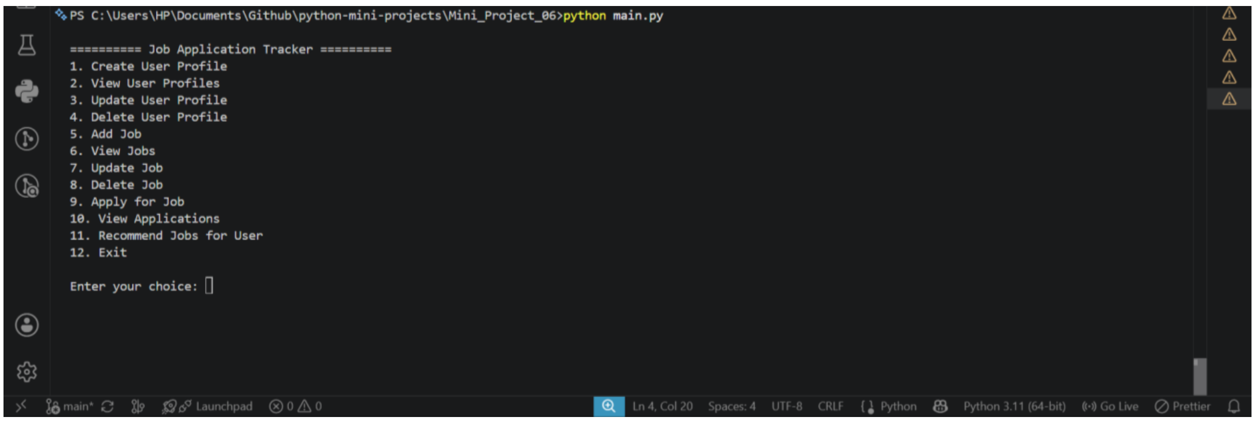
Display Results

↓

End

7. Output Screenshots and Explanation

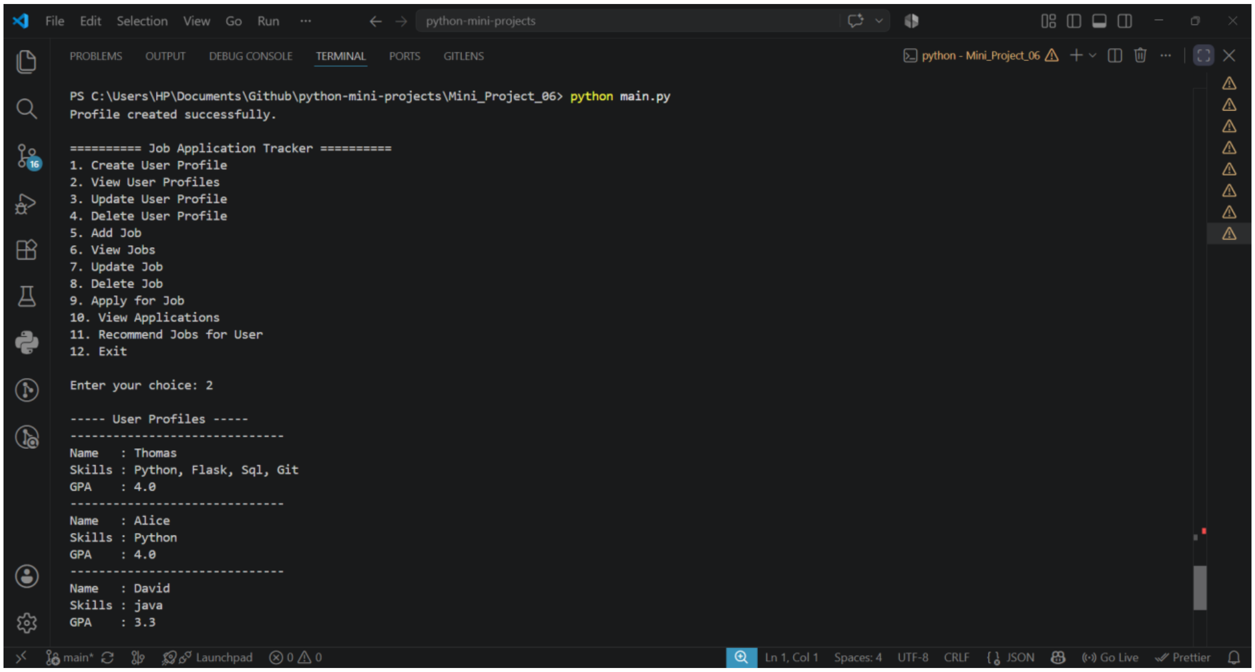
Screenshot 1: Main Menu



Explanation

The application starts by displaying a menu-driven interface with options to manage user profiles, job listings, and job applications. Users can choose different operations by entering the corresponding menu number.

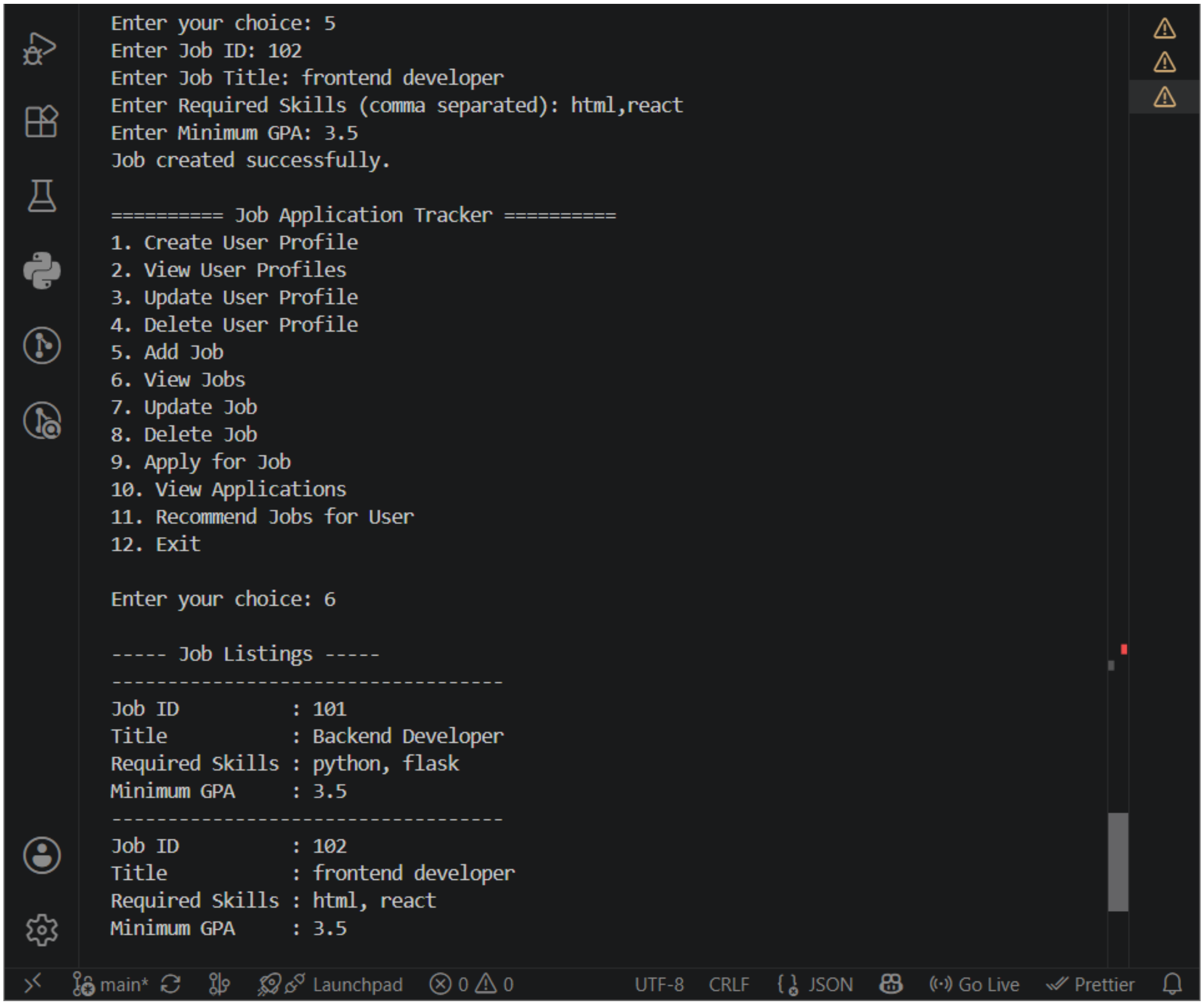
Screenshot 2: User Profile Management



Explanation

The application allows users to create, view, update, and delete user profiles. Each profile stores the user's name, skills, and GPA, which are later used to validate eligibility during the job application process.

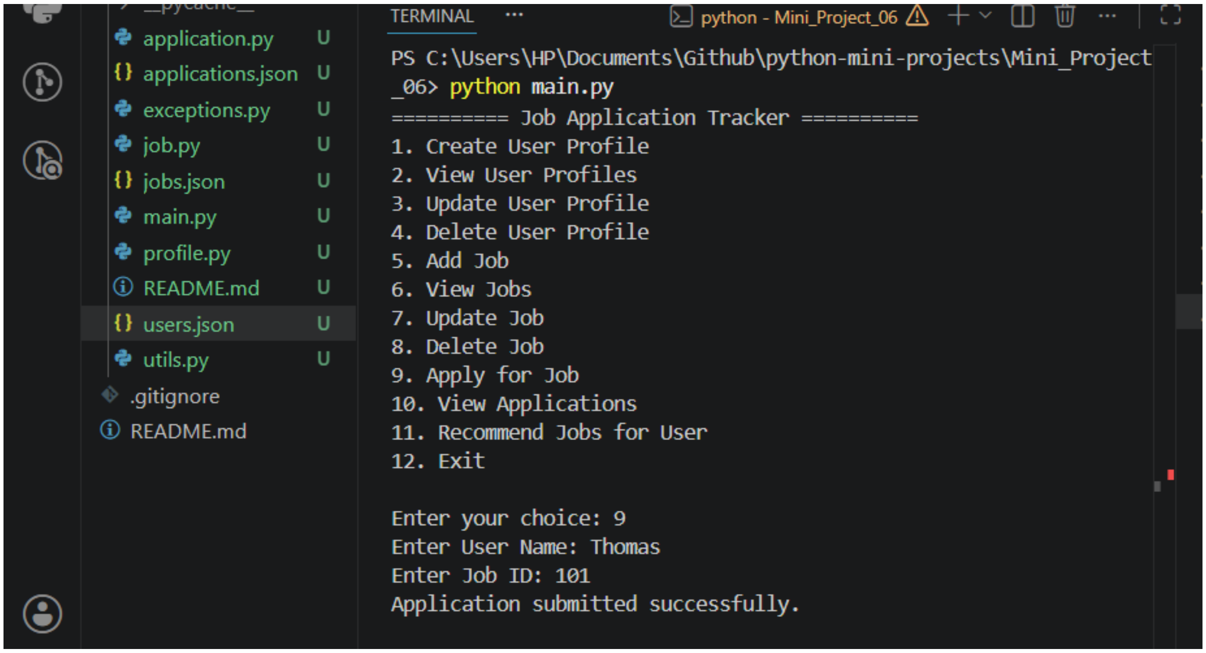
Screenshot 3: Job Management



Explanation

The application provides functionality to add, view, update, and delete job listings. Each job record contains a Job ID, job title, required skills, and the minimum GPA required for eligibility.

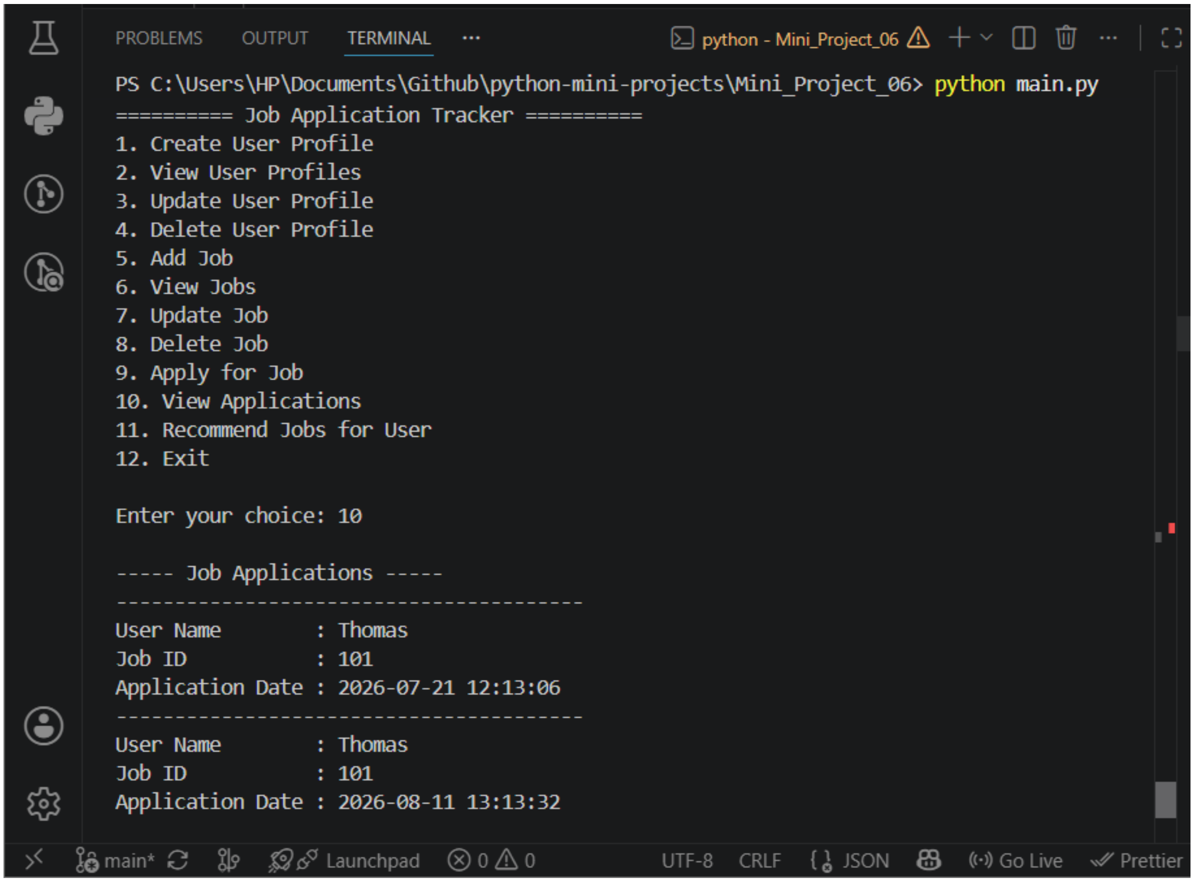
Screenshot 4: Successful Job Application



Explanation

After validating the user's GPA and required skills, the application successfully submits the job application and records the application date and time. The submitted application is stored in the applications.json file.

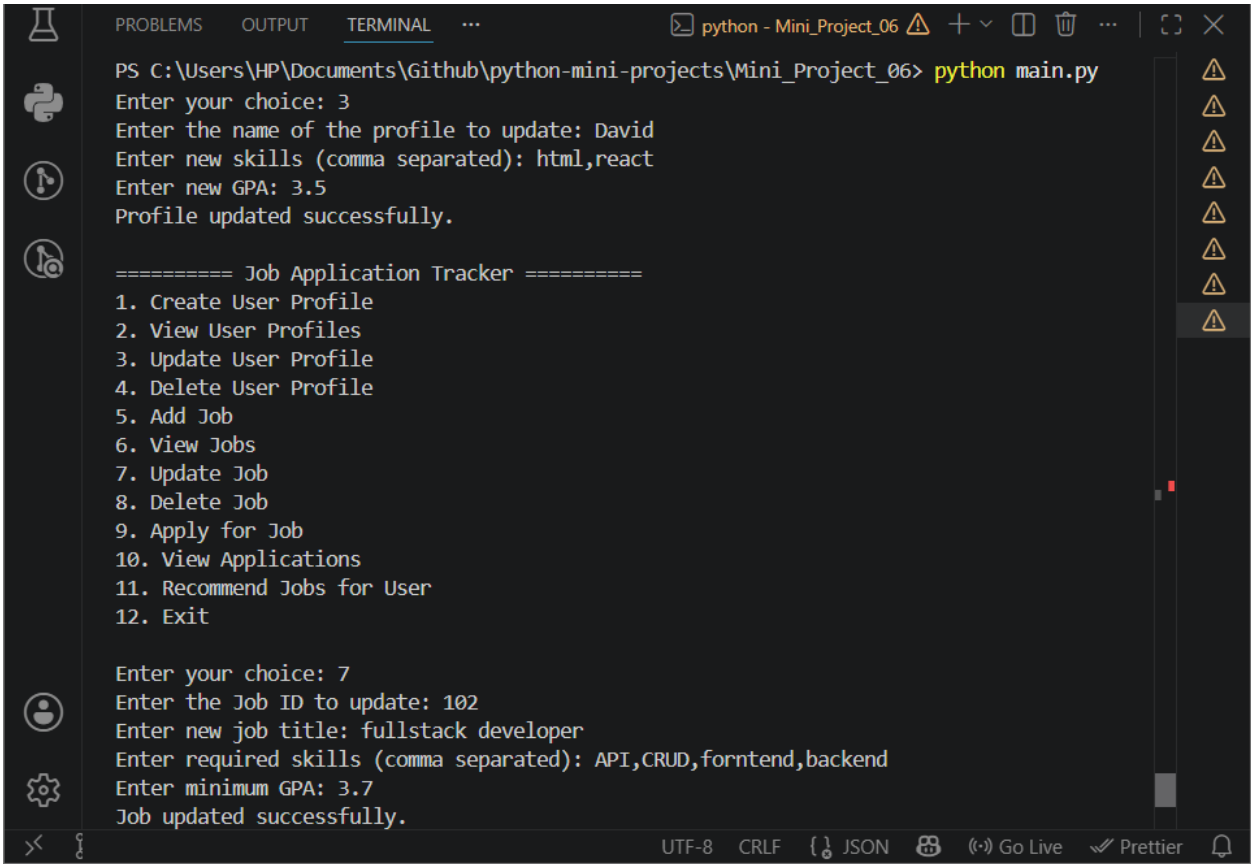
Screenshot 5: Viewing Applications



Explanation

The application displays all submitted job applications along with the applicant's name, Job ID, and application date. This allows users to review previously submitted applications.

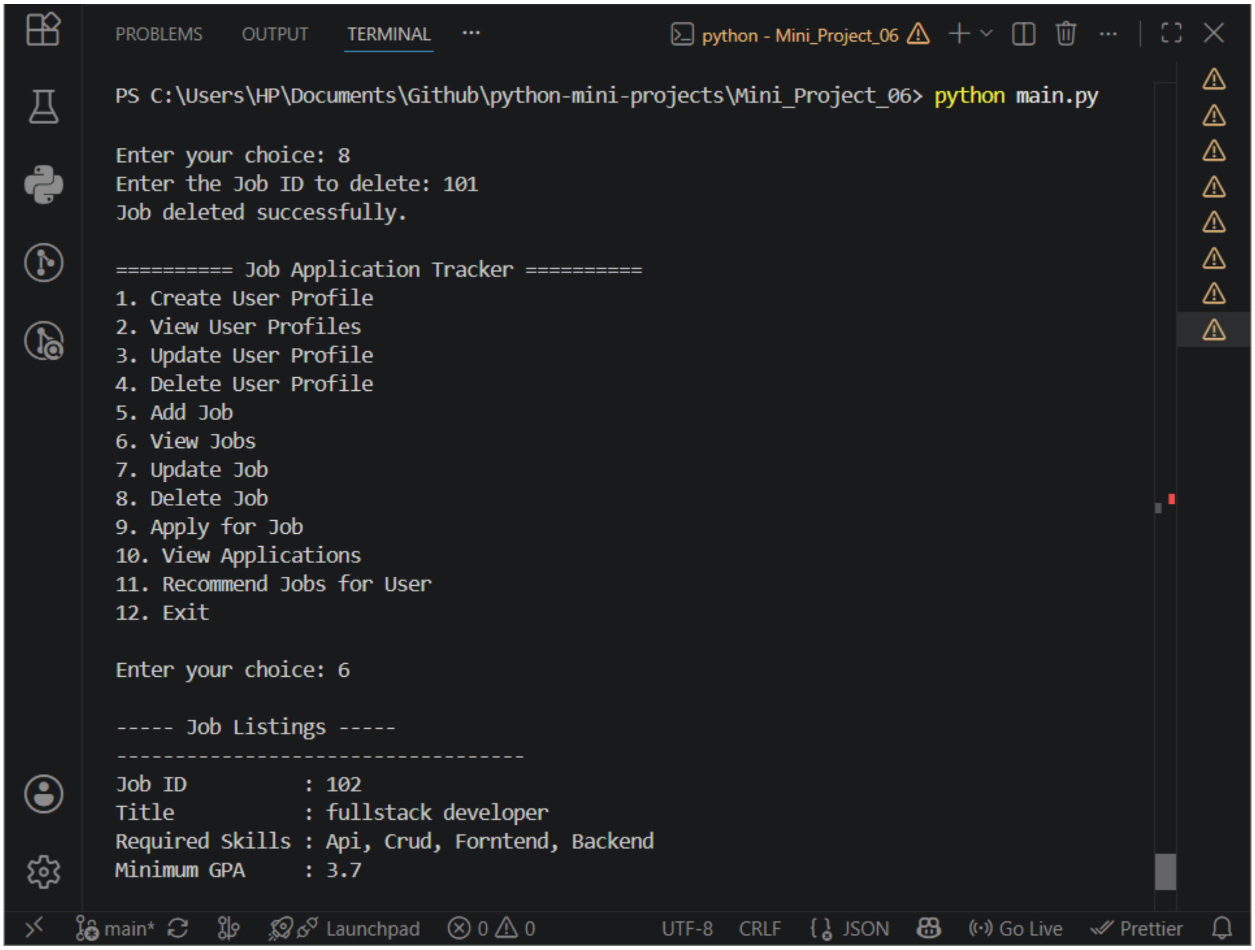
Screenshot 6: Updating User and Job Information



Explanation

The application allows users to modify existing profile details and job information. The updated data is saved to the corresponding JSON files, ensuring that future operations use the latest information.

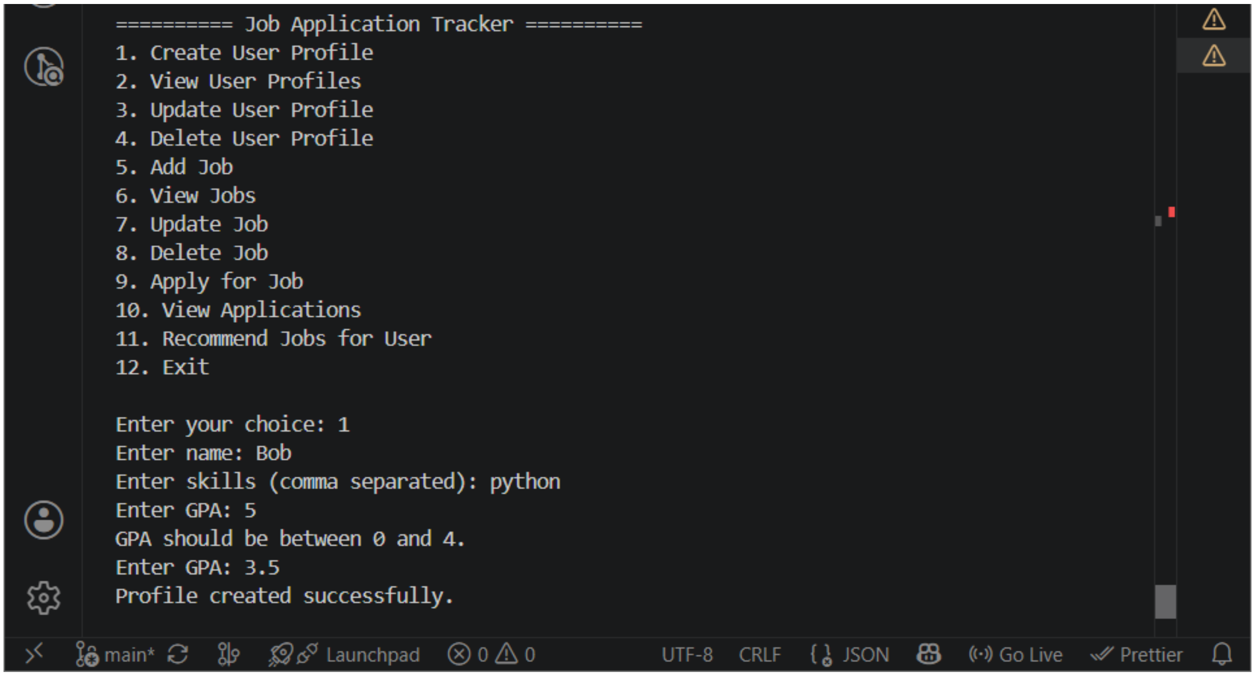
Screenshot 7: Deleting User and Job Records



Explanation

The application provides options to remove user profiles and job listings. After deletion, the corresponding records are permanently removed from the JSON files, while previously submitted application records remain unchanged.

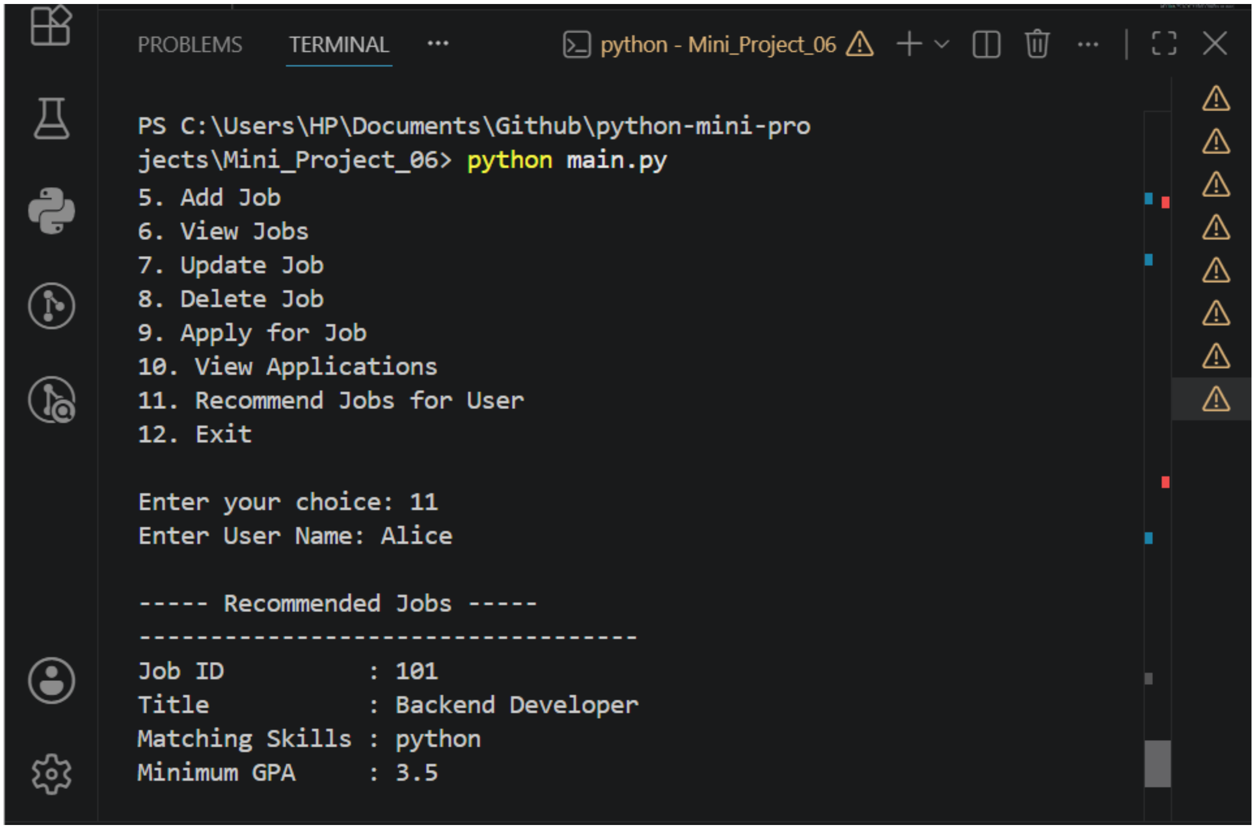
Screenshot 8: Exception Handling



Explanation

The application validates user input and handles exceptions gracefully. It displays appropriate messages for scenarios such as duplicate user profiles, duplicate job IDs, insufficient GPA, missing required skills, invalid menu choices, or attempting to apply for non-existent jobs or user profiles. These validations ensure reliable and error-free execution.

Screenshot 9: Job Recommendations



Explanation:

The application recommends available jobs based on partial skill matching. A recursive function checks the user's skills against the required skills of each job and displays suitable job recommendations.

8. Learning Outcomes

After completing this project, I learned how to:

- Develop modular Python applications using multiple modules
- Apply Object-Oriented Programming concepts
- Implement CRUD operations using JSON files
- Create and use custom exceptions
- Validate user eligibility using GPA and skills
- Work with date and time using the datetime module

- Implement recursive functions for job recommendations
- Perform partial skill matching
- Build menu-driven console applications
- Organize code into reusable and maintainable modules

9. Conclusion

The Job Application Tracker successfully demonstrates the implementation of Object-Oriented Programming concepts, modular programming, JSON file handling, and custom exception handling in Python. The project provides an efficient way to manage user profiles, job listings, and job applications while ensuring proper validation of eligibility criteria. It also strengthens the understanding of CRUD operations, file handling, exception handling, and menu-driven application development. Overall, this project offers practical experience in designing modular, maintainable, and user-friendly Python applications.